```
    webTestClient.get().uri("/quotes/random")
      .exchange()
      .expectStatus().is2xxSuccessful()
      .expectBody(Quote.class);
  }

  @Test
  void whenRandomQuoteByGenreThenReturn() {
    webTestClient.get().uri("/quotes/random/FANTASY")
      .exchange()
      .expectStatus().is2xxSuccessful()
      .expectBody(Quote.class)
      .value(quote -> assertThat(quote.genre()).isEqualTo(Genre.FANTASY));
  }
}
```

That's it for the implementation. Next we'll execute the auto-tests and run the application on the JVM.

RUNNING AND TESTING ON THE JVM

So far, Quote Service is a standard Spring Boot application, no different from any other applications we built in the previous chapters. For example, we can run the autotests with Gradle and ensure that it behaves correctly. Open a Terminal window, navigate to the project's root folder, and execute the following command:

```
$ ./gradlew test
```

We can also run it on the JVM or package it as a JAR artifact. From the same Terminal window, execute the following command to run the application:

```
$ ./gradlew bootRun
```

Feel free to verify that the application works correctly by calling the endpoints exposed by Quote Service: